Lecture 16 - Binary Search Trees and AVL Trees

Foundations of Computer Science (4CC505)

Dr Sam O'Neill

I highly recommend that you use the e-lecture mode on Visualgo.net for both

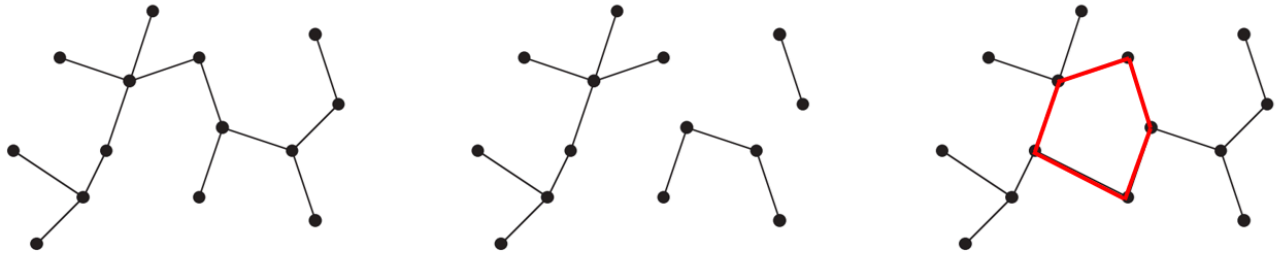
- Binary Search Trees
- AVL Trees

Trees

Formally:

Free Tree is a connected, acyclic, undirected graph.

- Connected - can reach all nodes from any node in graph
- Acyclic - does not contain any cycles
- Undirected - can move along edge either way. e.g. 2 way street!



(a) Free Tree

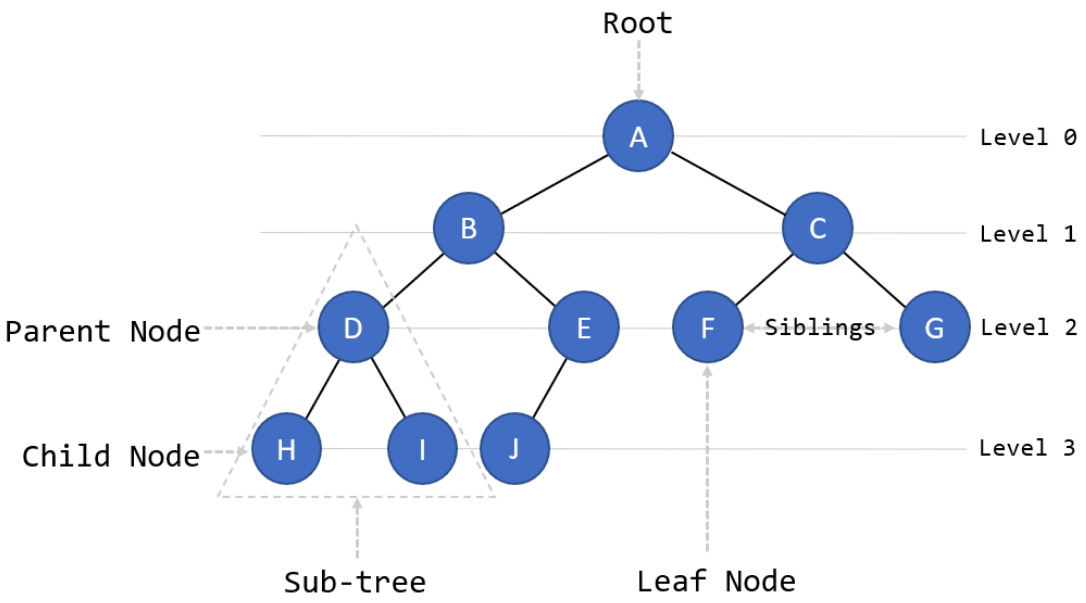
(b) Not Connected

(c) Contains a cycle

Binary Tree

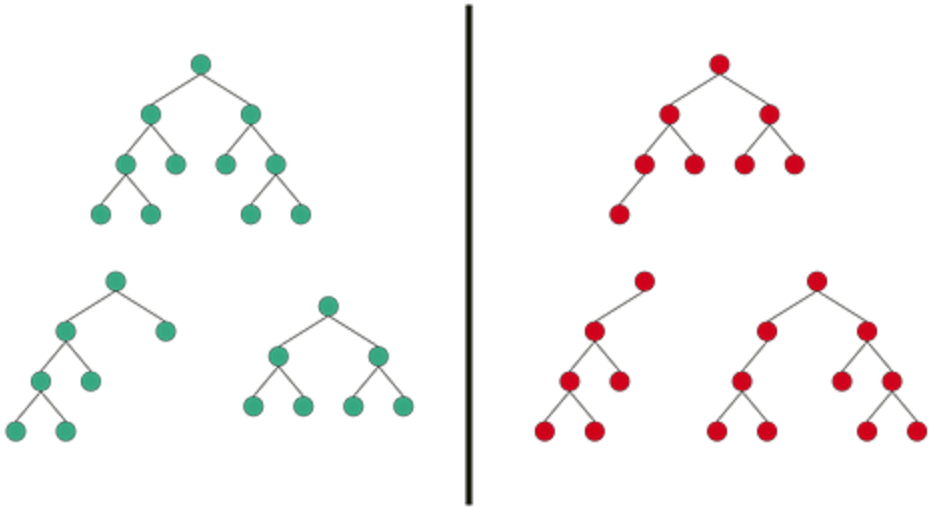
A binary tree has:

- a root node
- each node has at most two children
  - a left child
  - a right child



Full Binary Tree

- **Full** - Every node has 0 or 2 children

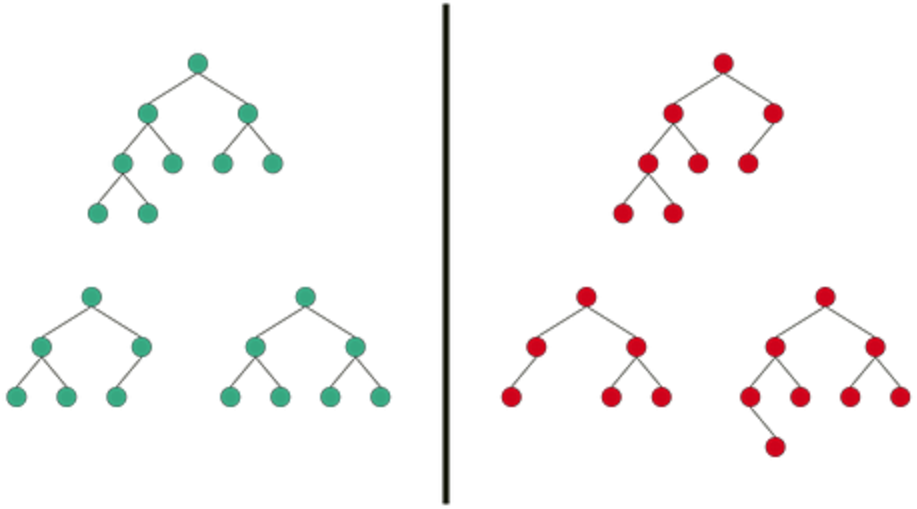


https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Green examples are **full** binary trees.

Complete Binary Tree

- **Complete** - Every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible.
- Height is $\lfloor \log(n) \rfloor$ (n is number of nodes).

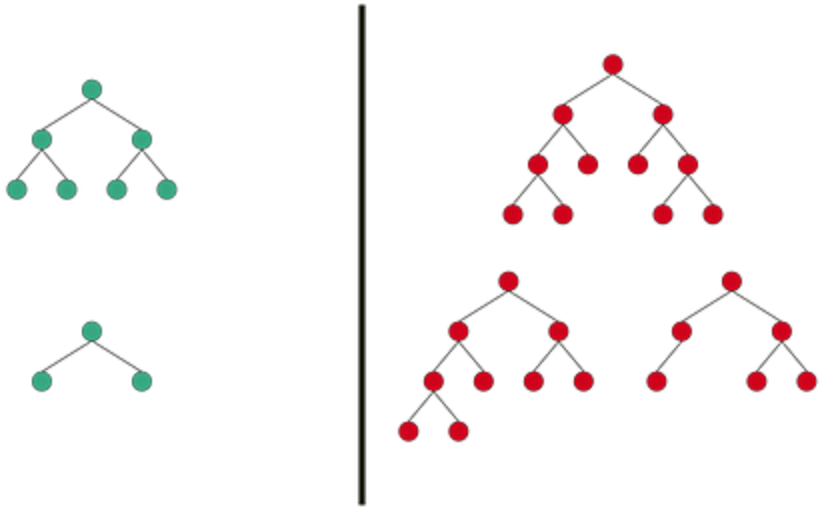


https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Green examples are **complete** binary trees.

Perfect Binary Tree

- **Perfect** - All interior nodes have two children and all leaves have the same depth or same level.
- Height is $\lfloor \log(n) \rfloor$ (n is number of nodes).

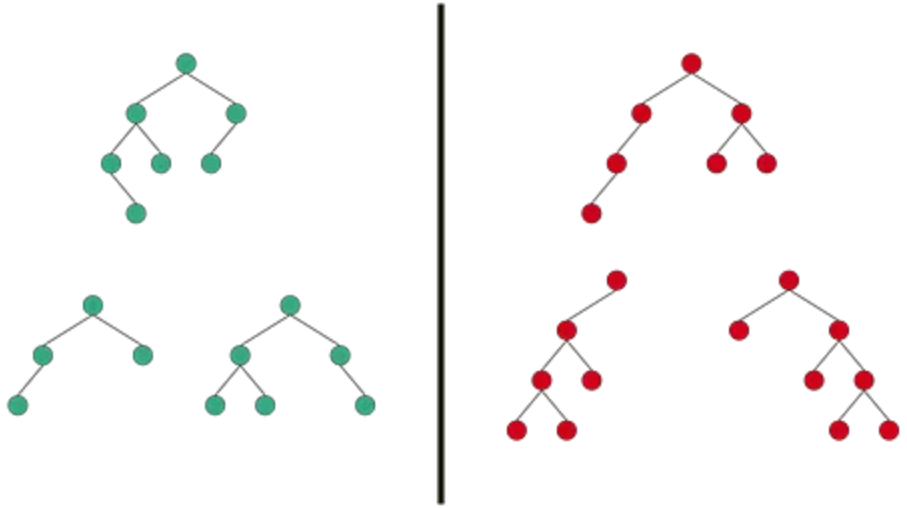


https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Green examples are **perfect** binary trees.

Balanced Binary Tree

- **Balanced** - Left and right subtrees of every node differ in height by no more than 1

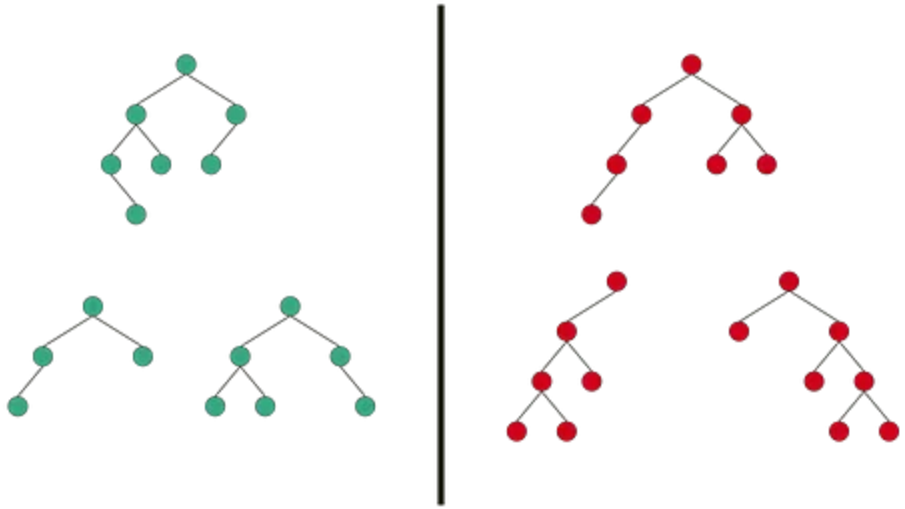


https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Green examples are **balanced** binary trees.

Degenerate (Pathological) Binary Tree

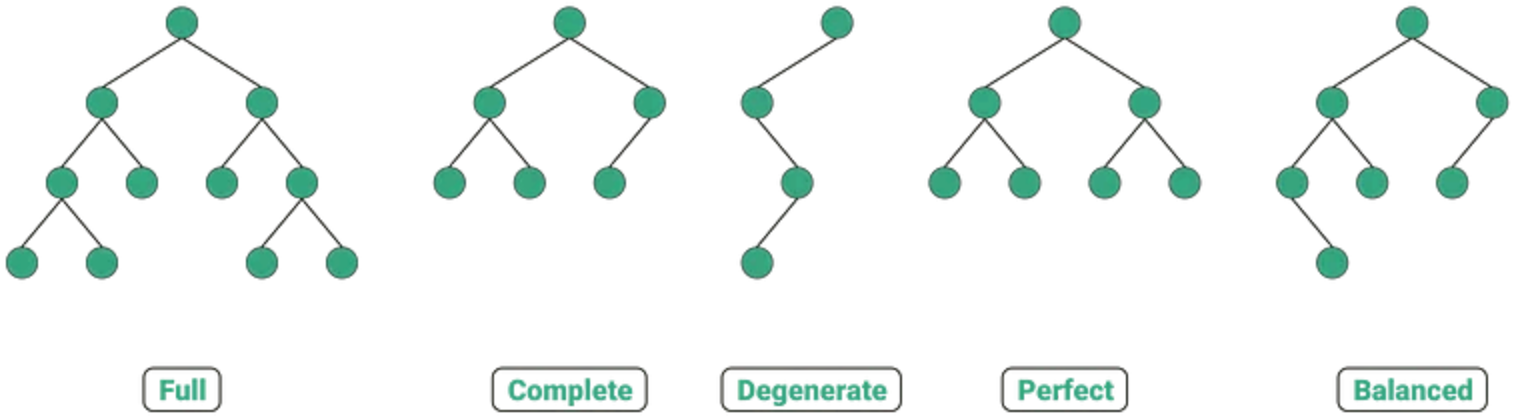
- **Degenerate** - Each parent node has only one associated child node



https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Green examples are **degenerate** binary trees.

Binary Tree Types



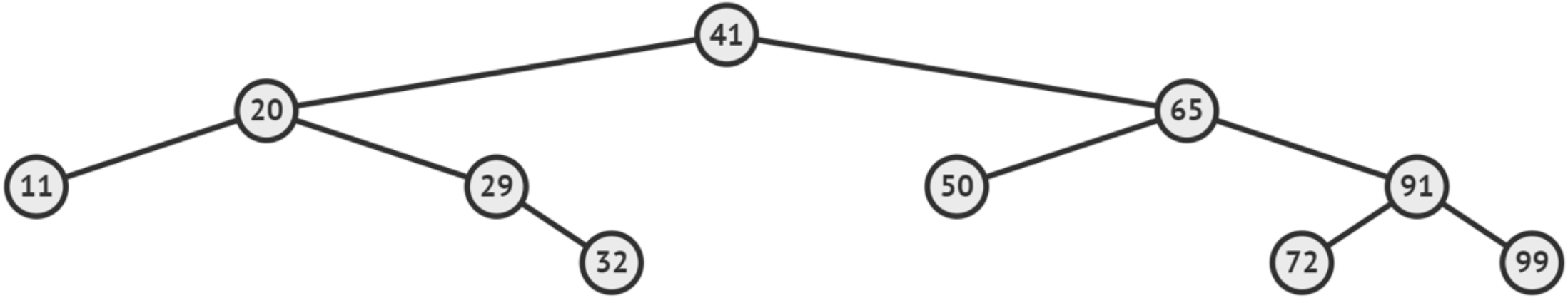
https://towardsdatascience.com/5-types-of-binary-tree-with-cool-illustrations-9b335c430254

Binary Search Trees (BST)

- Binary Tree T
- Node x contains
  - a value - $x.key$
  - left-child node - $x.left$
  - right-child node - $x.right$
  - parent node - $x.parent$
- Each node satisfies the **binary search tree (BST) property**
  - If y is a node in the **left subtree** of x then $y.key \leq x.key$
  - If y is a node in the **right subtree** of x then $y.key \geq x.key$

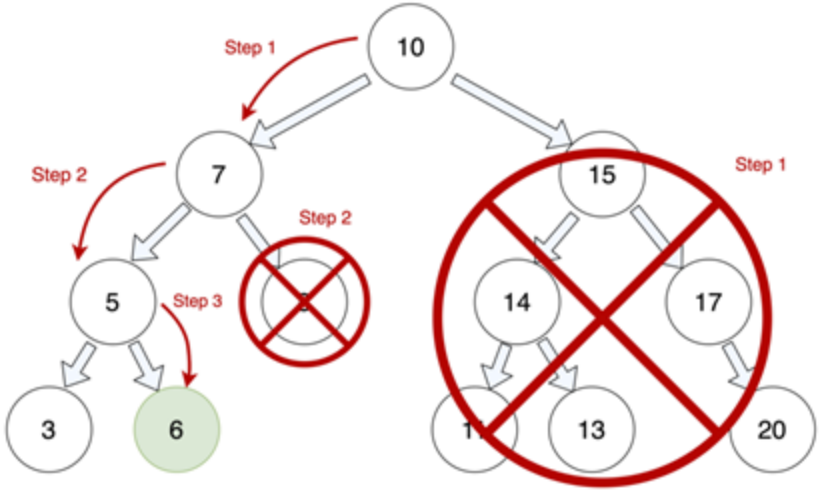
Binary Search Tree (BST) Property

- Each node satisfies the **binary search tree (BST) property**
  - If y is a node in the **left subtree** of x then $y.key \leq x.key$
  - If y is a node in the **right subtree** of x then $y.key \geq x.key$



Why?

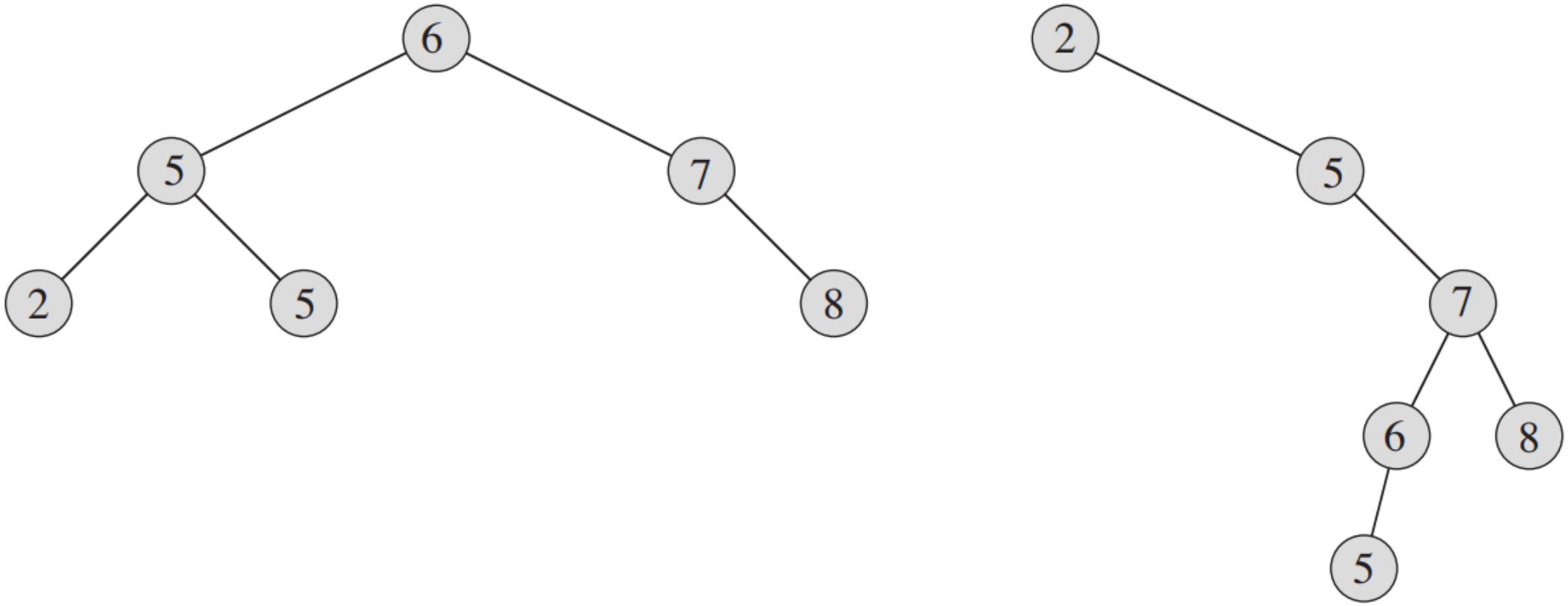
- We can store order!
- Fast search - average-case is $O(\log(n))$



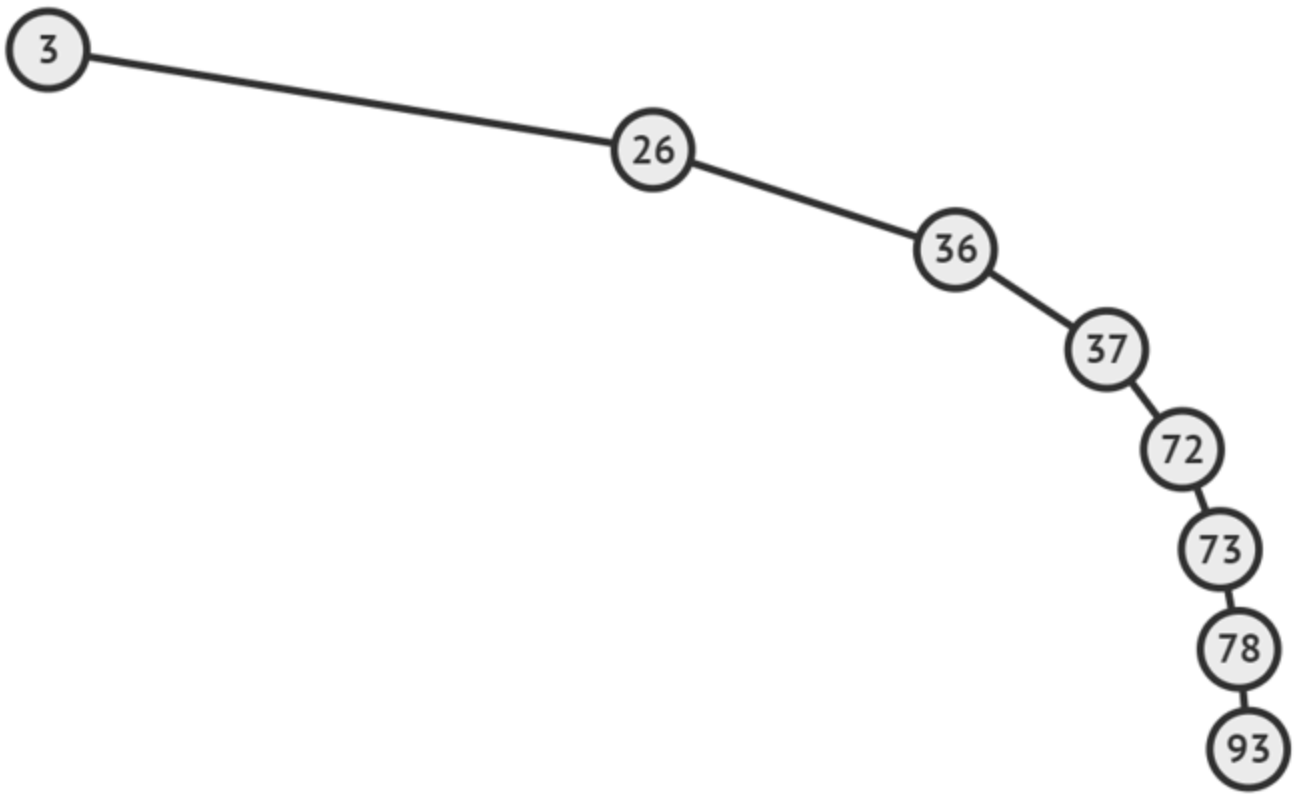
Here the **BST** has $n = 12$ nodes, so takes $\lfloor \log_2(12) \rfloor = 3$ steps to find 6.

Not a Unique Representation

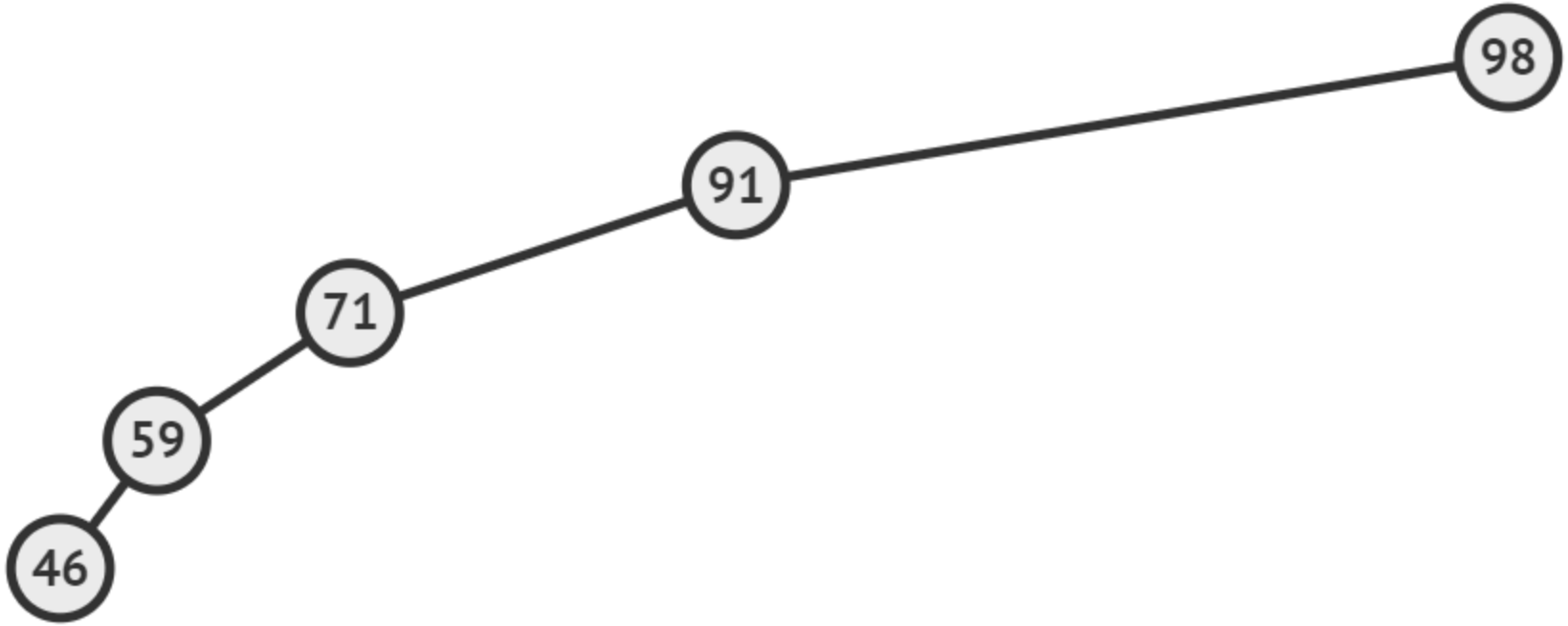
It is not guaranteed that the BST represents things uniquely.



Skewed Right



Skewed Left



Binary Search Tree Operations

- `search()`
- `minimum()`
- `maximum()`
- `insert()`

There are also two other operations:

- `delete()`
- `predecessor()`
- `successor()`

These are more complicated and we leave you to explore these. Chapter 12 (BST) of CLRS - Intro to Algorithms explains all of these. Also check out Visualgo.net - BST e-Lecture Mode.

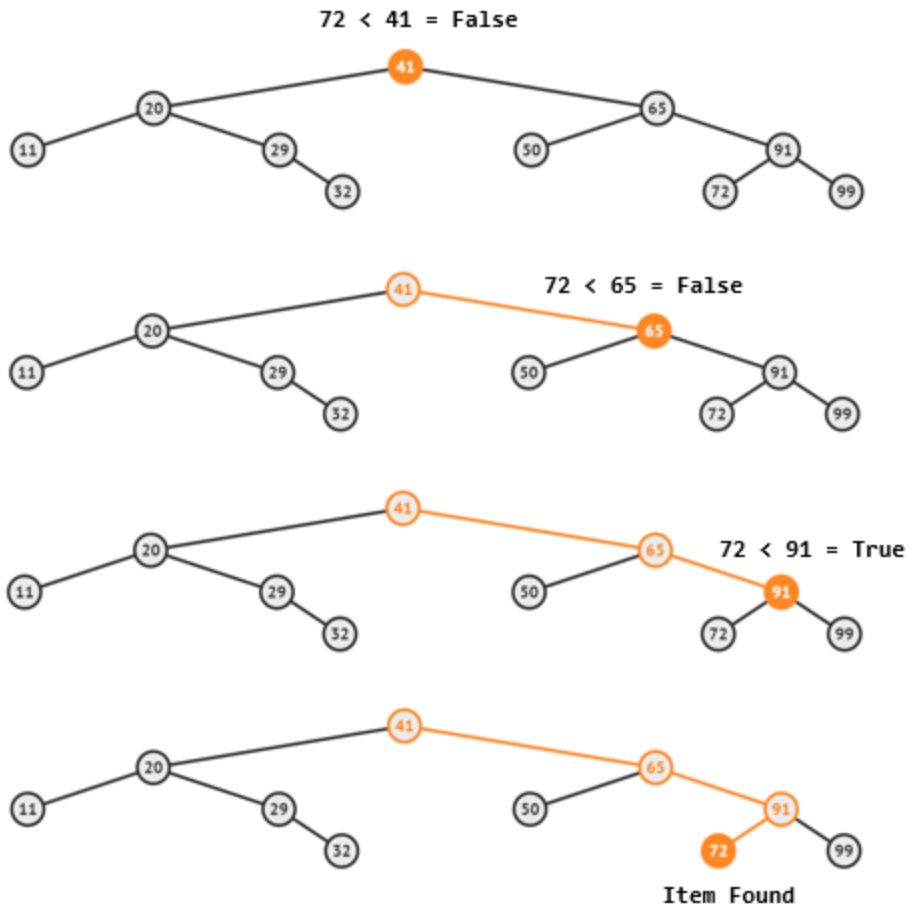
Search

```
def search(T, k):

  x = T.root # set node to root node of tree

  # repeat until x found or x None
  while x != None and k != x.key
    # if key less than nodes key
    if k < x.key:
      # set x to left-child
      x = x.left
    else:
      # set x to right-child
      x = x.right

  # return either found node or None
  return x
```



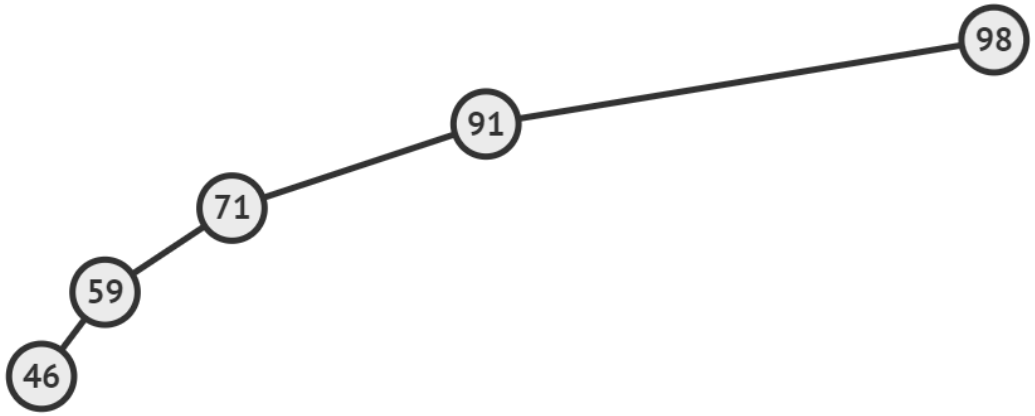
Minimum

Simple, just keep going left!

```
def minimum(T):
  x = T.root # set node to root node of tree

# repeat until x found or x None
while x.left != None
    x = x.left

return x
```



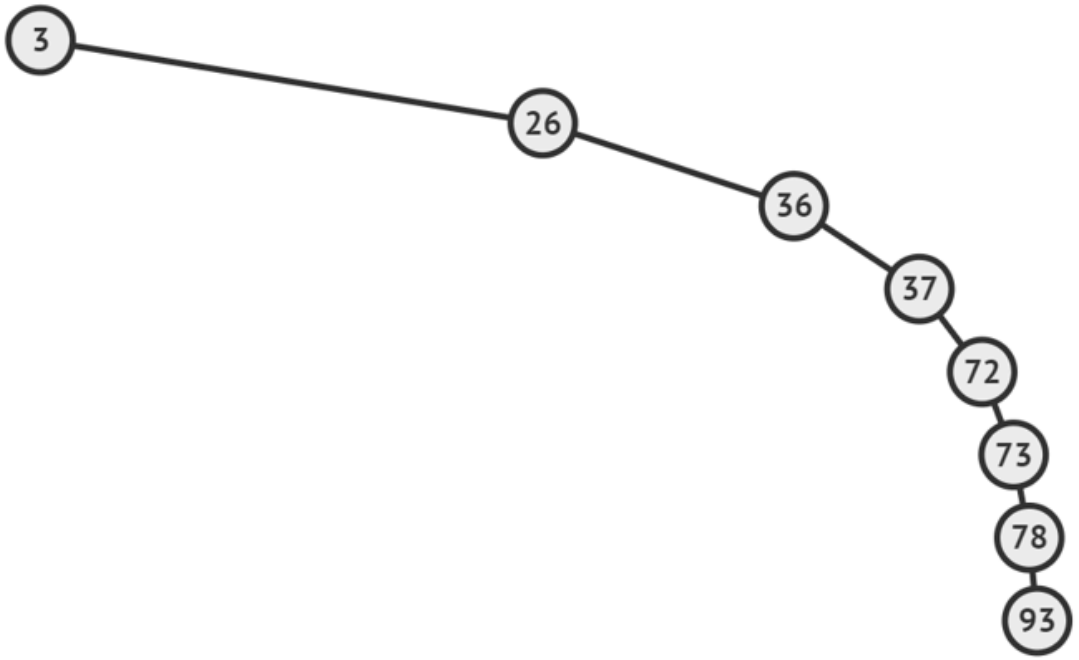
Maximum

Simple, just keep going right!

```
def maximum(T):
  x = T.x # set node to root node of tree

# repeat until x found or x None
while x.right != None
    x = x.right

return x
```



Insertion

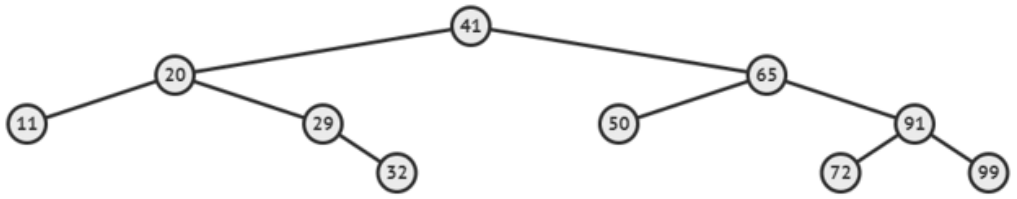
```
def insert(T, z):
  """ Insert node x into BST T """
  y = None
  x = T.root # set node to root node of tree

# find parent (insertion node) and set y = parent
while x != None
  y = x
  if z.key < x.key:
    x = x.left
  else:
    x = x.right

# set node z's parent to y
z.parent = y

if y == None:       # tree is empty
  T.root = z
elif z.key < y.key: # insert to left
  y.left = z
else:               # insert to left
  y.right = z

return x
```



Deletion

This requires transplanting a node - which is more complicated.

Please see chapter 12 (BST) of CLRS - Intro to Algorithms for an explanation.

You can also use the e-lecture mode on Visualgo.net.

Time Complexities

Because we can have skewed left and skewed right binary search trees.

We may have to visit n nodes when searching, inserting and deleting.

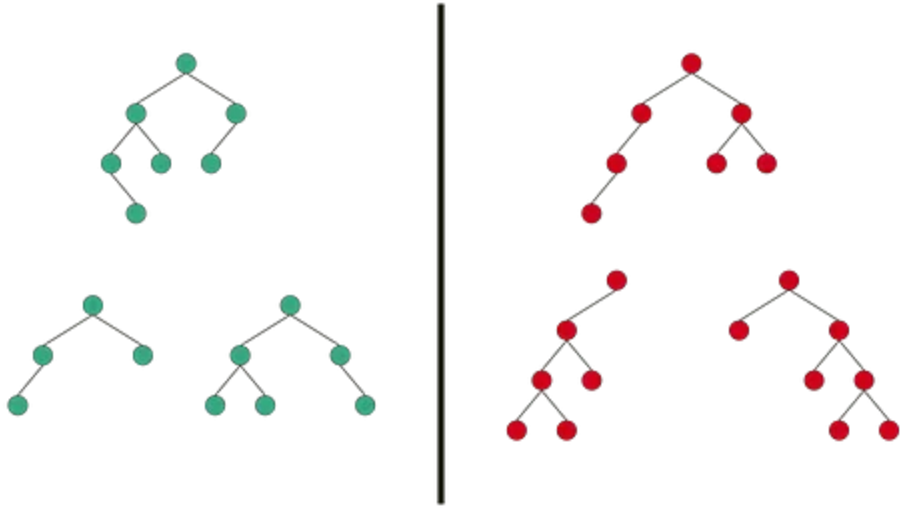
Therefore worst-case is $O(n)$.

	Average-case	**Worst-case**
`search()`	$O(\log(n))$	$O(n)$
`maximum()`	$O(\log(n))$	$O(n)$
`minimum()`	$O(\log(n))$	$O(n)$
`delete()`	$O(\log(n))$	$O(n)$
Space	$O(n)$	$O(n)$

Can we do better?

Yes, if we keep the binary search tree **balanced**.

- **Balanced** - Left and right subtrees of every node differ in height by no more than 1



AVL Trees

There are a number of self-balancing binary search trees.

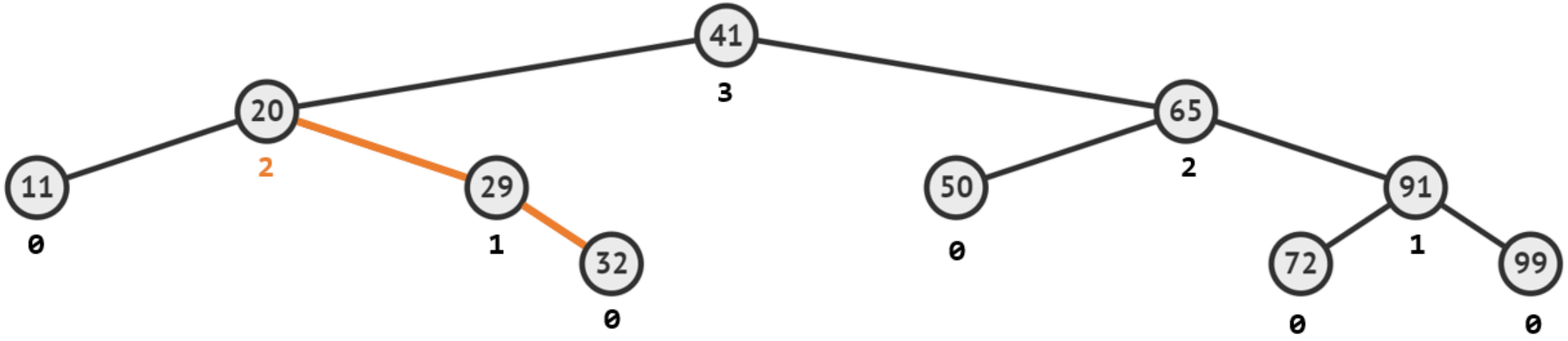
We will explore AVL Trees.

- Named after inventors Adelson-Velsky and Landis
- Similar to another type called red-black trees

Height of a Node (Vertex)

The **height** of a node x - $x.height$ is defined as:

The number of edges in the path to node x's deepest leaf node.



AVL Tree Invariant

Define the balance factor of node x as:

$$BF(x) := x.left.height - x.right.height$$

Then the AVL Tree invariant is:

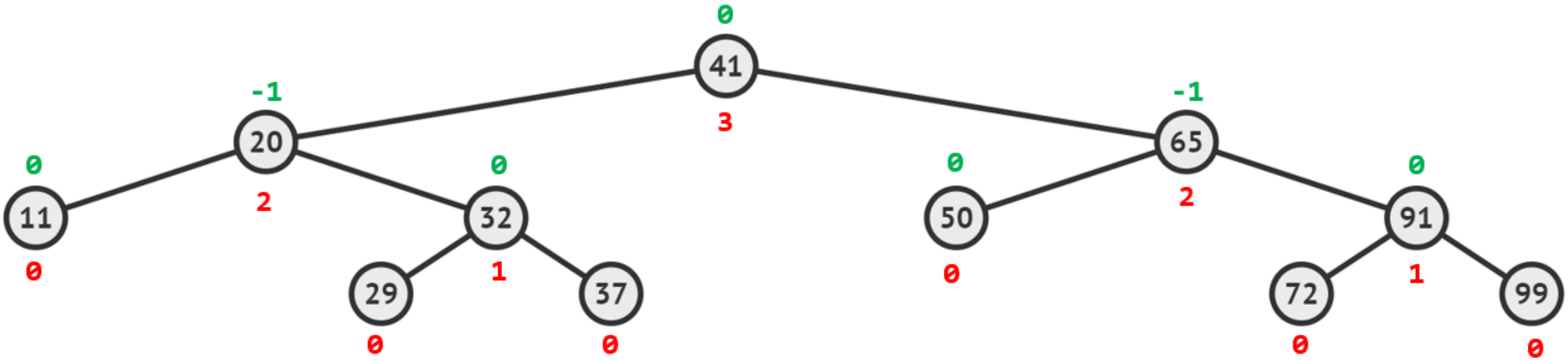
$$BF(x) \in \{-1, 0, 1\}$$

which holds for every node x.

AVL Tree Invariant - Holds

Here the AVL Tree invariant holds.

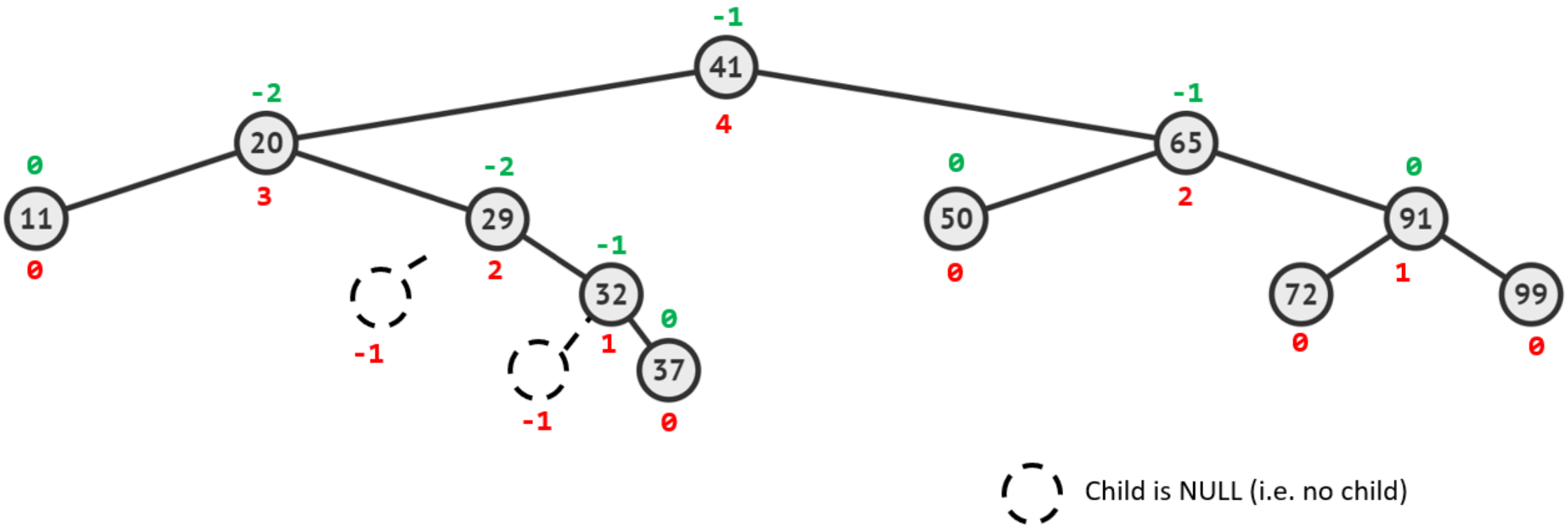
- Height in **red**
  Balance Factor in **green**
  - $BF(x) := x.left.height - x.right.height$



AVL Tree Invariant - Does Not Hold

Here the AVL Tree invariant **DOES NOT** hold.

- Height in **red**
  Balance Factor in **green**



Height of an AVL Tree

The height of an AVL tree is just the height of the root node.

It can be proved (beyond this module) that an BST that satisfies the AVL tree invariant with n nodes has a height of $O(\log(n))$.

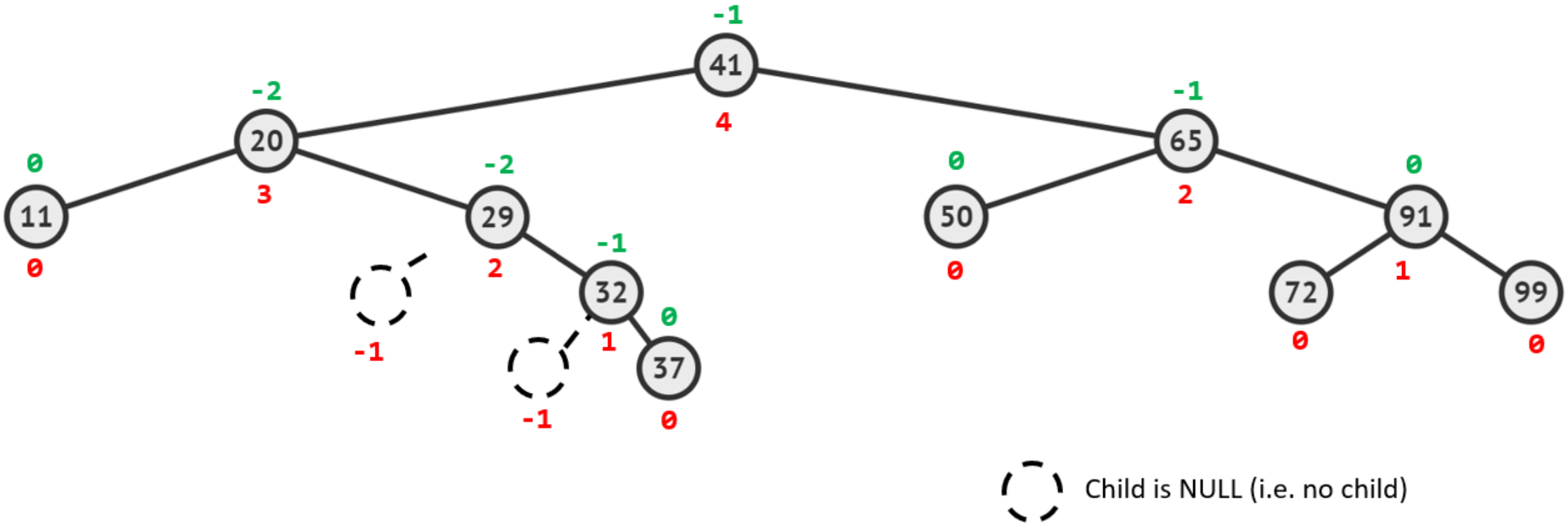
Thus all BST operations like search, insert, delete... will now be worst-case $O(\log(n))$.

However, we now need to check and maintain the AVL tree invariant after insertion and deletion!

Insertion

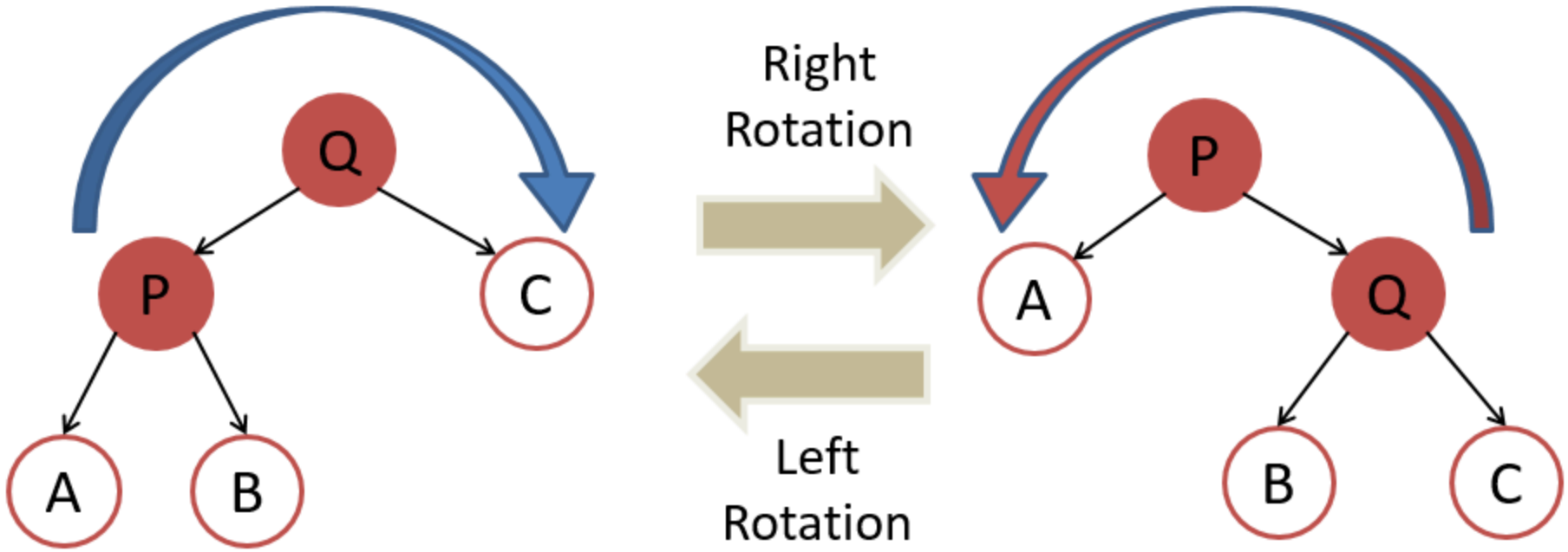
Insert as before.

Issue! Tree may be unbalanced. e.g. 37 inserted into the BST

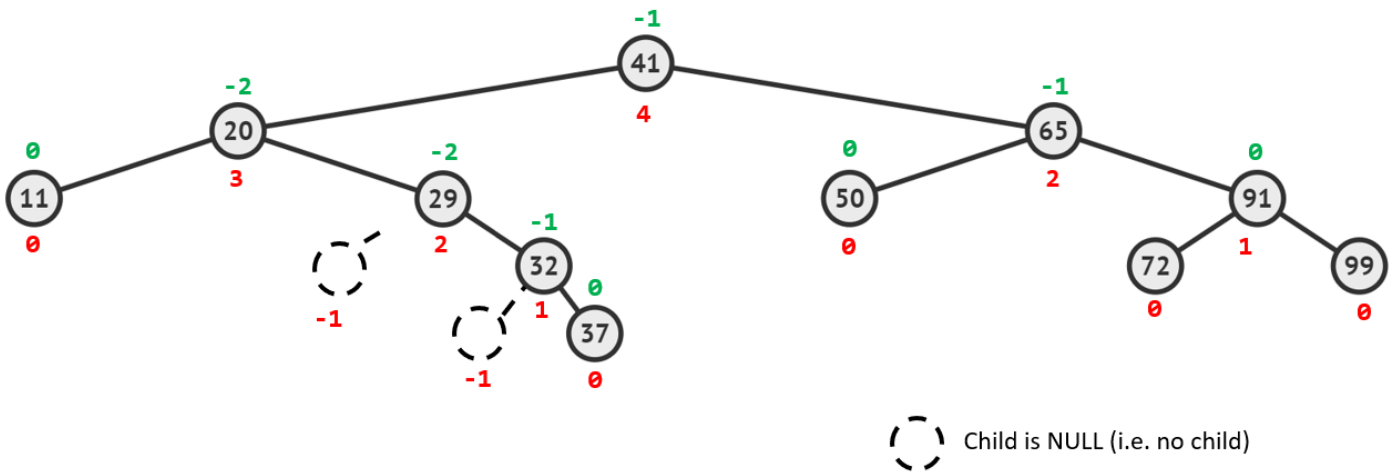


Rotations

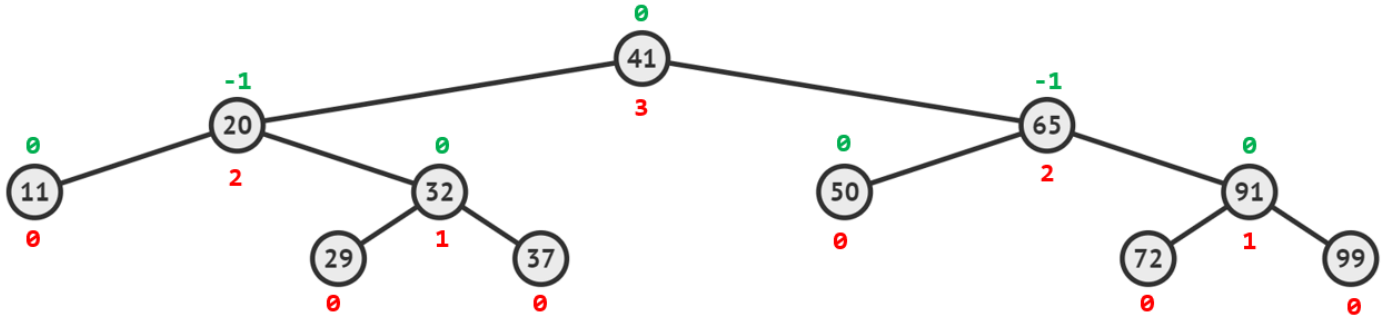
We introduce the idea of a left-rotation and a right-rotation.



Example



Perform a left-rotation on 29 and 32. Tree is then balanced (below).



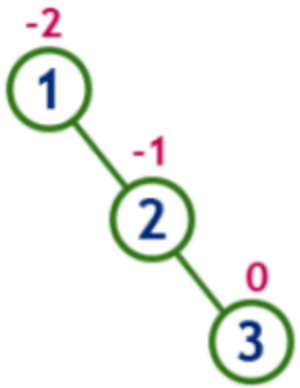
Four Possible Unbalanced Cases

- Left Rotation (Left Left)
- Right Rotation (Right Right)
- Left Right Rotation (Left Right)
- Right Left Rotation (Right Left)

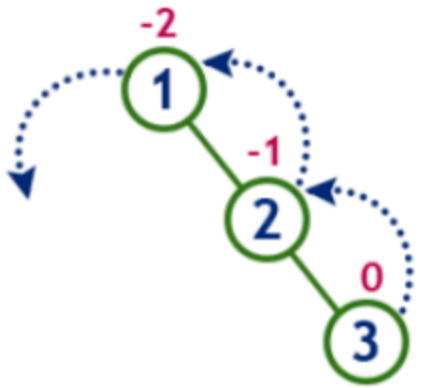
Four Possible Unbalanced Cases

Left Rotation (Left Left)

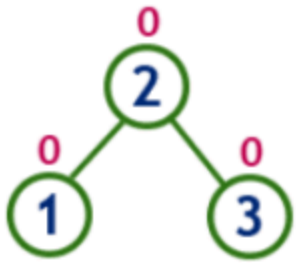
insert 1, 2 and 3



Tree is imbalanced



To make balanced we use LL Rotation which moves nodes one position to left

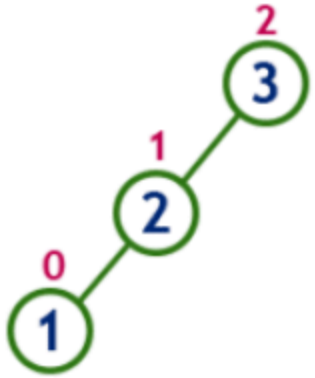


After LL Rotation Tree is Balanced

Four Possible Unbalanced Cases

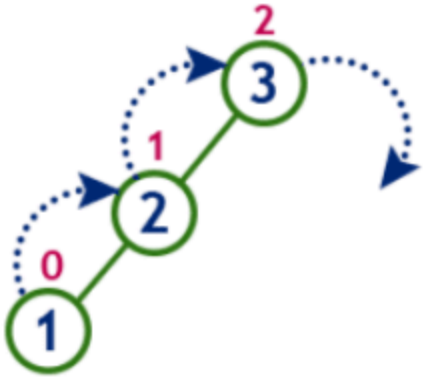
Right Rotation (Right Right)

insert 3, 2 and 1

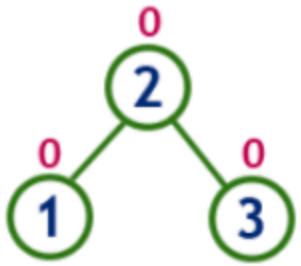


Tree is imbalanced

because node 3 has balance factor 2



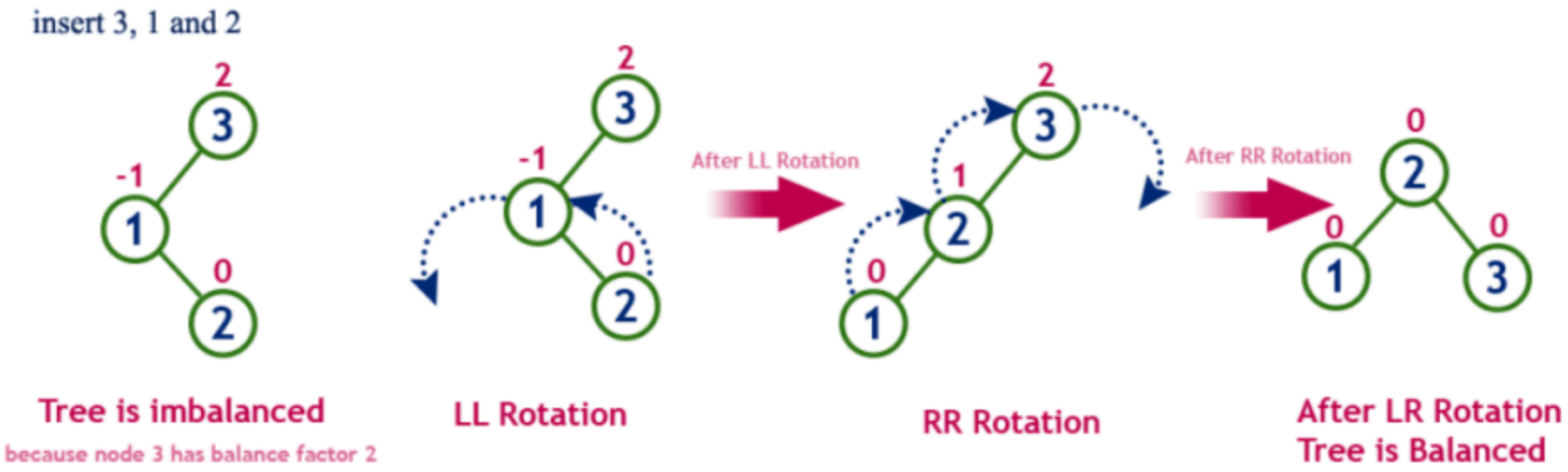
To make balanced we use RR Rotation which moves nodes one position to right



After RR Rotation Tree is Balanced

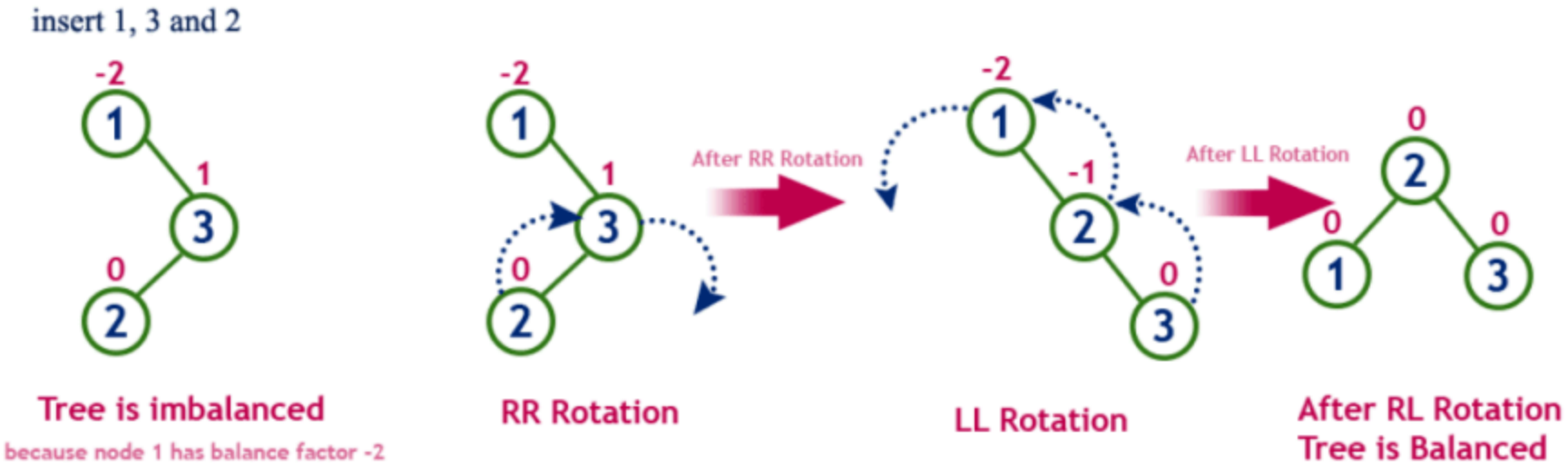
Four Possible Unbalanced Cases

Left Right Rotation (Left Right)



Four Possible Unbalanced Cases

Right Left Rotation (Right Left)



Insertion Algorithm

To insert a node z.

1. Insert as normal for a BST
2. Walk up the AVLTree from the insertion point. For every node:

- Check balance factor, if 2 or -2 then:
  - Rebalance using 1 of the 4 rotation cases

Please see chapter 12 (BST) of CLRS - Intro to Algorithms for an explanation.

You can also use the e-lecture mode on Visualgo.net.

Deletion Algorithm

To remove a node z.

1. Remove as normal for a BST
2. Walk up the AVL Tree from the insertion point. For every node:

- Check balance factor, if 2 or -2 then:
  - Rebalance using 1 of the 4 rotation cases

Please see chapter 12 (BST) of CLRS - Intro to Algorithms for an explanation.

You can also use the e-lecture mode on Visualgo.net.

Summary

- Binary Search Tree (BST) stores nodes in order
- AVL Tree is a self-balancing BST
  - We do this using rotations

Worst-case Summary

	BST	AVL Tree
`search()`	$O(n)$	$O(\log(n))$
`maximum()`	$O(n)$	$O(\log(n))$
`minimum()`	$O(n)$	$O(\log(n))$
`delete()`	$O(n)$	$O(\log(n))$
Space	$O(n)$	$O(n)$

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. Introduction to algorithms. MIT press.

Visualgo.net